

JobMatch: An Agentic Multi-Role Platform for Explainable Job–Candidate Matching

First Author^{1*}

^{1*}Department of Computer Science, University Placeholder, City, Country.

Corresponding author(s). E-mail(s): author@example.edu;

Abstract

Job seekers and recruiters often face the same frustration: ranked lists with no clear reason behind them, and no separation between who owns a profile, who owns a job posting, and who is allowed to change a match. We present JobMatch, a multi-agent recruitment platform in which Candidate and Employer agents maintain role-specific profiles and shared embeddings, while a read-only Matchmaking agent scores and explains candidate–job pairs. Agents coordinate through an event bus; when a resume or job description changes, the owning agent updates vector representations and notifies the Matchmaking layer to refresh stale rankings. Separate candidate, employer, and admin portals keep save, apply, and shortlist decisions with human users.

The Matchmaking agent combines semantic similarity, skill overlap, and experience, compensation, and remote compatibility into a fixed composite score with structured explanations—assistive ranking, not autonomous hiring. We evaluate offline on 30 resumes, 15 job postings, and 47 graded relevance pairs using macro-averaged P@5, R@5, and nDCG@5 at $K=5$. Compared methods include TF–IDF and BM25 baselines, dense retrieval, reciprocal rank fusion, learned fusion, and cross-encoder reranking. Multimodal soft-embed and learned fusion reach nDCG@5 of 0.969 and 0.968 in committed artifacts; composite ablation reaches 0.942 under a documented weight vector. Cross-encoder reranking reduces nDCG@5 by 0.108 at ~ 141.7 ms/query and remains disabled in the default portal. Hard-negative mining yields 150 pairs with zero label conflicts; a synthetic fairness audit flags 6/10 fabricated pairs—an engineering check, not demographic validation.

Benchmark drivers and regression tests reproduce reported metrics. We document multi-agent architecture, implementation, quality metrics, and results as an integrated research prototype, not a validated production hiring system.

Keywords: multi-agent systems, recruitment platforms, explainable matching, job-candidate ranking, hybrid retrieval

1 Introduction

For many job seekers, finding work still feels like shouting into a crowd. They rewrite the same resume for dozens of roles, wait weeks without feedback, and rarely learn why one posting surfaced in their results while another vanished after a single click. When the process does move forward, it often moves on the employer’s timetable—not the candidate’s—leaving people to guess what mattered most: their skills, their location, their salary expectations, or something they never saw in the job ad.

Recruitment systems were built to help organizations sort large applicant pools, but the experience on both sides remains uneven. Candidates see opaque lists and generic rejections. Recruiters and hiring managers see dashboards that prioritize speed over clarity: filters that drop promising profiles, rankings that change without explanation, and little visibility into how a job requirement maps to a person’s actual background. Trust erodes when neither party can answer a simple question—*why this match, and why now?*

The cost of getting filtering wrong is measured in time and money, not just frustration. Every unsuitable interview consumes calendar space, interviewer attention, and candidate hope. Every missed fit—someone qualified who never appeared on a shortlist—can mean a longer vacancy, a rushed hire, or lost talent to a competitor. At scale, small errors in matching compound into weeks of delay and substantial recruiting spend, even before anyone discusses compensation or start dates.

A broader vision of computing may help reframe what hiring tools could become. In his writing on education and artificial intelligence, Sal Khan describes a future in which personal AI assistants walk alongside individuals—helping them understand options, prepare materials, and navigate complex decisions rather than replacing human judgment. That idea transfers naturally to the job market: a job seeker deserves an assistant that represents *their* goals, history, and constraints, not merely a form field on an employer’s website.

We take that vision seriously by giving each side of the market its own software agent. On the candidate side, a **client-side agent** accepts resumes and profile updates, helps organize skills and experience into a structured record, and keeps that record current as the person edits their materials. On the employer side, an **employer-side agent** accepts job descriptions and hiring requirements, structures them into a comparable record, and maintains them as roles evolve. Neither agent exists in isolation: a candidate’s story and an employer’s needs only become meaningful when they can be compared fairly, explained clearly, and updated when either party changes direction.

Collaboration among agents is therefore the design center, not an afterthought. If candidate information and job information live in separate silos with no shared language, both portals revert to blind search. If a single monolithic program owns every edit and every score, users cannot tell who changed what or why a ranking

shifted. JobMatch instead assigns clear ownership—candidates control candidate state, employers control job state—and introduces a dedicated **matchmaking agent** that reads both sides, scores possible pairings, and returns ranked recommendations with reasons attached. When a resume or job description changes, the owning agent updates its record and notifies the matchmaking layer so suggestions stay fresh.

Together, these pieces form **JobMatch**, a multi-agentic job-matching system implemented as a research prototype with separate web experiences for candidates, employers, and administrators. The system parses resumes and job descriptions into structured states, builds comparable representations of people and roles, and routes matching work through the matchmaking agent. Users remain in charge: they review parsed fields, inspect why a pairing scored as it did, and explicitly choose to save, apply, or shortlist—the software suggests; it does not hire on anyone’s behalf. We evaluate the matching core offline on a labeled demo corpus (30 resumes, 15 jobs, 47 graded relevance pairs) using standard quality metrics and reproducible benchmark comparisons; we report numbers only from committed artifacts and do not claim production hiring effectiveness.

This paper makes the following contributions:

- A **candidate/client-side agent** that processes resumes and CVs and maintains candidate state.
- An **employer-side agent** that processes job descriptions and maintains job state.
- A **matchmaking engine** that uses semantic matching and ranking to produce ordered recommendations.
- A **multi-agent communication flow** in which agents share relevant state and data when profiles change.
- A **complete UI and application layer** demonstrating end-to-end candidate, employer, and admin workflows.
- An **evaluation** using quality metrics and benchmark comparisons on a fixed labeled corpus.

The remainder of the paper is organized as follows. Section 2 reviews recruitment systems, AI agents in hiring, and semantic matching. Section 3 presents the architecture of the multi-agent system. Section 4 describes implementation details. Section 5 defines quality metrics and the evaluation protocol. Section 6 reports results and discussion. Section 7 concludes and outlines future scope.

2 Literature Review

Recruitment technology spans applicant tracking, job boards, and recommender-style matching. We group prior work into four themes and state how JobMatch differs: it is a *multi-agent* platform with shared representations and human-in-the-loop portals, not a standalone semantic search engine.

2.1 Recruitment systems and hiring workflows

Enterprise hiring stacks combine document ingestion, applicant tracking, and recruiter workflows. Candidates experience opaque ranking when systems expose only a score or a yes/no filter; recruiters struggle when they cannot see which requirements drove a shortlist. JobMatch addresses this gap at the *workflow* level: candidate and employer agents own their respective data, a matchmaking agent explains pairings, and users retain explicit control over applications and shortlists (§3–§4).

2.2 AI agents in hiring and multi-agent coordination

Multi-agent systems assign specialized roles that communicate through messages or events [1]. In hiring, a natural decomposition separates candidate-side state, employer-side state, and read-only scoring. Prior agentic hiring proposals often emphasize autonomous negotiation or interview bots; JobMatch instead documents *bounded* agents that synchronize profiles and rankings while humans approve actions. Agent communication in our system uses an in-process event bus; profile updates invalidate match caches without cross-agent mutation of stored records.

2.3 Semantic matching and hybrid retrieval

Job-candidate matching builds on information retrieval: lexical methods (TF-IDF, BM25) [2, 3], dense embeddings [4], and rank fusion [5]. Hybrid pipelines combine semantic similarity with structured skill overlap and constraint checks. Much published work treats matching as a single retrieval formula evaluated on static corpora. JobMatch integrates these signals inside a composite ranker with structured explanations, and reports lexical, dense, fusion, and reranking variants on the *same* labeled corpus (Tables 3, 5).

2.4 Explainability, calibration, and fairness

Explainable ranking methods attribute scores to input features [6]; calibration maps scores to probability scales [7]. Fairness concerns in hiring include disparate impact across groups and sensitivity to non-qualification text. JobMatch attaches rule-structured explanations at scoring time and supports optional calibration and feedback boosts (disabled in the default portal). We report a synthetic counterfactual audit only (Table 11) and do not claim demographic fairness certification.

2.5 Limitations of current approaches and our positioning

Existing systems often (i) hide ranking logic from one or both sides of the market, (ii) collapse profile editing and scoring into one service, or (iii) evaluate retrieval in isolation from deployable workflows. JobMatch contributes an integrated research prototype: role-aware agents, shared snapshots, explainable composite ranking, and reproducible offline evaluation tied to the same code paths that power the portals. We do not assert state-of-the-art hiring accuracy; the contribution is traceable multi-agent design plus measured ranking behavior on a small demo corpus.

Fig. 1: Multi-agent recruitment architecture. Candidate and Employer agents own profile state and embeddings; they publish update events on a shared bus. The Matchmaking agent consumes immutable snapshots, scores pairs, and returns ranked explanations to role portals. The UI layer never writes directly to vector stores.

3 Architecture of the Multi-Agent System

JobMatch organizes hiring support around three collaborating agents and a shared representation layer. Figure 1 shows the overall layout; Figures 2 and 3 show how candidates and employers interact with their respective agents. Implementation details (APIs, libraries, parameters) appear in §4; scoring formulas and evaluation metrics in §5.

3.1 Design principles

Three principles govern the architecture. **Separation of ownership:** only the Candidate agent writes candidate snapshots; only the Employer agent writes job snapshots. **Read-only matchmaking:** the Matchmaking agent scores and explains but does not edit profiles, apply on behalf of users, or send messages to employers. **Human-in-the-loop actions:** save, apply, contact, and shortlist operations flow through portal APIs and activity stores; ranking suggests, users decide.

3.2 The three agents

Candidate agent.. Represents the job seeker’s side of the market. When a user uploads or edits a resume, this agent parses assistive fields, normalizes skills, builds a versioned *snapshot*, computes a document embedding, and upserts the vector index. It publishes a profile-updated event so downstream components know prior rankings may be stale.

Employer agent.. Represents the hiring organization’s side. When a user posts or revises a job description, this agent registers a job snapshot with required and preferred skills, experience and compensation bounds, remote policy, and an embedding of the description text. It publishes job-updated events on the same bus.

Matchmaking agent.. Serves both portals. Given a candidate snapshot and a set of job snapshots (or the reverse direction), it runs the hybrid scoring core, optionally fuses multiple retrieval lists, attaches structured explanations, and returns ranked `MatchResult` rows. It subscribes to profile and job events to invalidate cached rankings but never calls write APIs on candidate or job stores.

3.3 Agent communication and the event bus

Agents communicate asynchronously through an in-process event bus (`agents/base.py`). Typical events include `CandidateProfileUpdated`, `JobProfileUpdated`, and `MatchCompleted`. When the Candidate agent finishes embedding a new resume version, it emits an update event; the Matchmaking agent

drops any cached ranking for that candidate. The Employer agent follows the same pattern for job edits.

This design keeps coupling low: agents expose HTTP endpoints through a gateway, but *coordination* happens via typed events rather than direct method calls across ownership boundaries. Administrators can inspect recent events in the admin console for debugging.

3.4 State ownership and shared representations

Each market side maintains two conceptual layers. A **profile** is the editable portal record (form fields the user sees). A **snapshot** is the immutable, versioned object passed to matching—skills, constraints, document hash, and embedding vector.

Ownership rules are strict. Candidate profiles and snapshots belong to the Candidate agent; job profiles and snapshots belong to the Employer agent. Feedback (save, dismiss, apply) and user activity logs live in SQLite and are owned by the gateway layer; they do not mutate snapshots unless the user submits a new profile version. The Matchmaking agent reads snapshots by identifier only.

Both agents write into a shared embedding space so that resume and job vectors are comparable. Skill aliases are canonicalized once so that lexical and set-based scores refer to the same vocabulary.

3.5 Vector-store updates and matchmaking flow

When a snapshot is registered or updated, the owning agent: (1) computes a fixed-dimensional embedding of canonical document text, (2) upserts the vector into a pluggable store (Chroma or Qdrant behind a **VectorStorePort** interface), (3) emits an update event.

The Matchmaking agent’s typical flow for a candidate seeking jobs is: (1) load the candidate snapshot for the authenticated user, (2) retrieve candidate jobs—either exhaustively (small pools) or via approximate nearest-neighbor search over job embeddings, (3) score each candidate–job pair with the configured strategy (portal default: composite), (4) attach explanations and return the top- K list.

Employer-side reverse matching mirrors the same steps with roles exchanged. Optional paths (ensemble fusion, cross-encoder rerank, calibration, feedback boost) exist in the codebase but are disabled in default portal requests (§4).

3.6 Role portals as the application layer

Candidates, employers, and administrators use separate route trees in one React application. Each tree enforces role-based access at the router and API layers. Portals call the gateway; they never invoke agent internals directly. Detailed UI flows, endpoints, and default parameters are documented in §4.

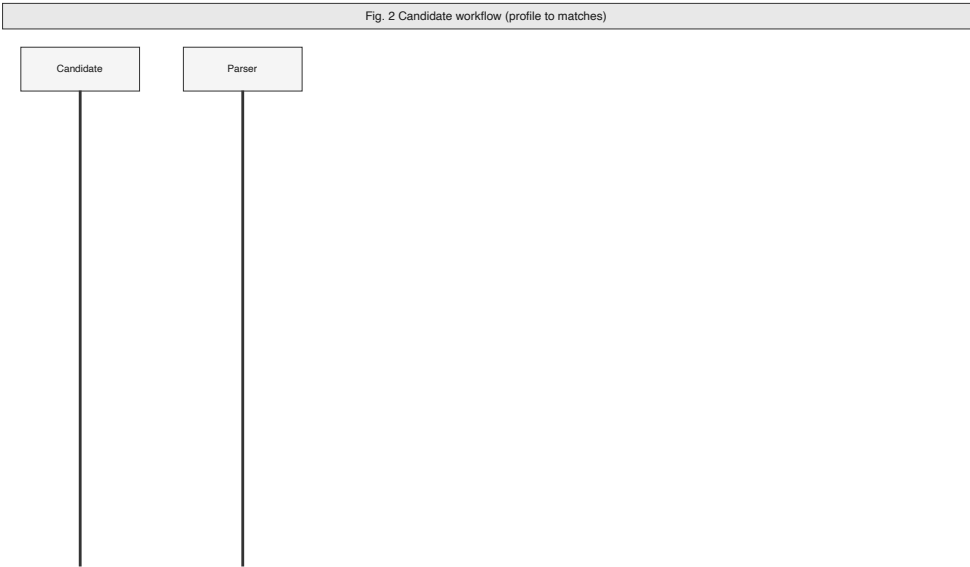


Fig. 2: Candidate workflow: resume upload, assisted parsing, profile confirmation, embedding registration, and ranked job discovery with explainable results.

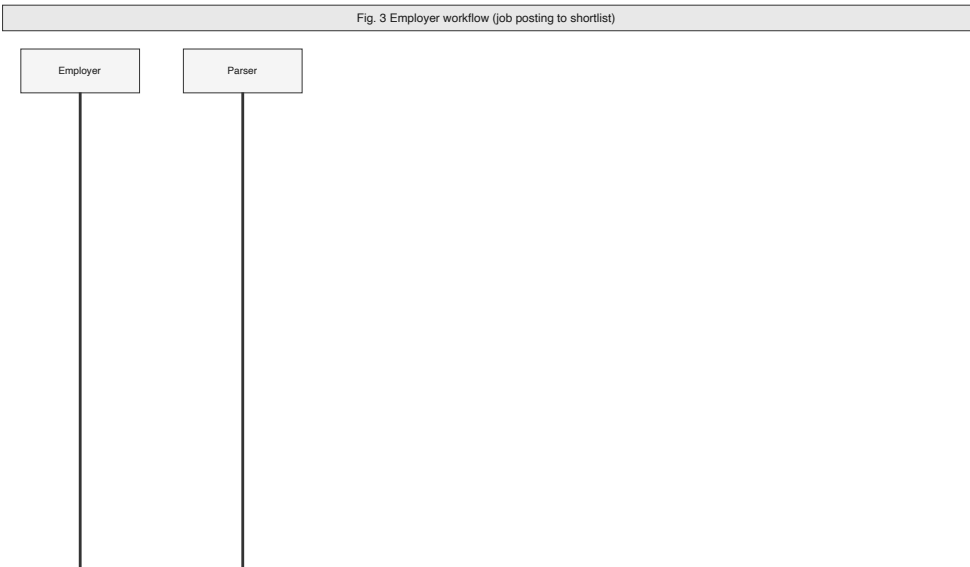


Fig. 3: Employer workflow: job description ingestion, posting confirmation, reverse candidate matching, and applicant review with structured explanations.

Fig. 4: Implementation of the matching pipeline: retrieval backends, composite scoring modules, optional fusion and calibration, and structured explanation generation consumed by portals.

3.7 Candidate and employer workflows

In both workflows, parsing is assistive—extracted fields populate forms and users confirm before an agent registers a snapshot. Matches appear with explanation panels; users explicitly choose next actions.

4 Implementation

This section describes how the architecture in §3 is built: technology stack, data structures, gateway APIs, matching pipeline modules, and portal behavior. Mathematical definitions of scores and metrics appear in §5.

4.1 Technology stack

JobMatch is a monolithic deployable application, not a microservice mesh. **Backend:** Python 3, FastAPI gateway (`backend/gateway/`), three domain agents (`backend/agents/`), matching core (`backend/core/`), SQLite for auth and activity, Chroma or Qdrant for vectors. **Frontend:** React SPA with role-specific layouts (`frontend/src/layouts/`), shared API client (`frontend/src/api/client.js`), and explainability components (`MatchExplainability`, `MatchDetailsDrawer`). **Embeddings:** `all-MiniLM-L6-v2` (384 dimensions) via Sentence-Transformers. **Benchmarks:** Python drivers under `backend/benchmarks/` with regression tests in `tests/benchmarks/`.

4.2 Data structures: profiles and snapshots

Candidate profile and snapshot.. The portal stores editable fields: name, skills, experience years, salary expectation, remote preference, summary, and canonical resume text. On save, the Candidate agent produces a snapshot with normalized skills, constraint scalars, document hash, version counter, and embedding vector. Only snapshots enter the Matchmaking agent.

Job profile and snapshot.. Employer postings store title, description, required and preferred skills, experience requirement, budget band, remote policy, and metadata. The Employer agent emits a parallel snapshot with embedding of the job description text.

Match results and explanations.. Each `MatchResult` includes identifiers, final score, strategy name, and an optional `MatchExplanation` object: matched and missing skills, fit rows for experience, compensation, remote, and semantic dimensions, plus per-component score breakdown for composite ranking.

Feedback and activity.. Match feedback (`save`, `dismiss`, `apply`) and broader user feedback (`contact`, `reject`) are event logs in SQLite. They do not alter embeddings unless the user submits a new profile version.

4.3 Gateway API surface

Table 1 summarizes primary flows; key endpoint groups follow.

Authentication.. POST `/auth/register`, POST `/auth/login`, POST `/auth/logout`, GET `/auth/me` with server-side session cookies.

Candidate routes.. GET/PUT `/candidates/me`, POST `/candidates/upload-resume`, quality and suggestion endpoints, saved jobs and applications.

Employer routes.. GET/POST `/jobs`, PUT `/jobs/mine/{job_id}`, description upload and parse endpoints, applicant lists.

Matching routes.. POST `/match/candidate-to-jobs`, POST `/match/job-to-candidates`, POST `/match/ensemble`, POST `/match/daily-batch`.

Admin and system routes.. GET `/agents/status`, GET `/agents/events/recent`, GET `/system/config`, GET `/system/fairness`, demo reset when enabled.

4.4 Portal defaults and request parameters

The frontend sends identical match defaults for candidate and employer views (DEFAULT_CANDIDATE_MATCH, DEFAULT_EMPLOYER_MATCH): `strategy=composite`, `topK=10`, `metric=cosine`, `skillsMode=jaccard`, `semanticWeight=0.7`, `fusionMode=fixed`, `applyConstraints=false`, `autoStrategy=false`, `useCalibration=false`, `useFeedbackBoost=false`, `explainMode=rules`.

Composite weights in `backend/core/scoring.py` are semantic 28%, skills 27%, title 10%, experience 15%, compensation 10%, remote 10%. API defaults in `MatchRequest` differ (e.g., `top_k=5`, `strategy=semantic`) when clients omit overrides; portals always pass composite explicitly.

4.5 Matching core modules

Retrieval.. `VectorStorePort` abstracts `upsert`, `search`, and `delete`. Production uses ANN search for large pools; the demo corpus often uses exhaustive scoring over 15 jobs. Lexical rankers (BM25, TF-IDF) and exact skill overlap exist for offline benchmarks only.

Composite and multimodal scoring.. `scoring.py` implements composite, multimodal weighted, semantic-only, and skill-only strategies. `matchmaking.scoring.py` orchestrates fusion modes, optional hard constraints, feedback boost, and Platt calibration when flags are enabled.

Fusion and routing.. `rff.py` merges multiple ranked lists (default $k=60$). `fusion.py` loads a learned logistic model from `data/models/fusion.json`. `strategy_router.py` selects strategy triples when `auto_strategy=true` based on skill count and summary length.

Table 1: Primary user-facing workflows by role ($K=10$ composite matching unless noted).

Role	Portal route	User goal	API / agent path
Candidate	<code>/candidate/onboarding</code>	Upload resume, review extraction, create profile	POST <code>/candidates/upload-resume</code> → PUT <code>/candidates/me</code> → Candidate agent registers snapshot + embedding
Candidate	<code>/candidate/profile</code>	Edit skills, experience, pay, remote prefs	GET/PUT <code>/candidates/me</code> ; quality via POST <code>/candidates/quality-check</code>
Candidate	<code>/candidate/matches</code>	View ranked jobs, filter, explain scores	POST <code>/match/candidate-to-jobs</code> (composite); profile readiness gate before search
Candidate	<code>/candidate/saved-jobs</code>	Saved jobs and applications	GET/PUT <code>/candidates/me/saved-jobs</code> ; POST <code>/candidates/me/applications</code>
Employer	<code>/employer/jobs</code>	Import JD, post/edit/close roles	POST <code>/jobs/upload-description</code> or <code>/parse-description</code> → POST <code>/jobs</code> or PUT <code>/jobs/mine/{id}</code> → Employer agent
Employer	<code>/employer/matches</code>	Rank candidates for a owned job	POST <code>/match/job-to-candidates</code> (composite); job ownership required
Employer	<code>/employer/applications</code>	Review applicants per posting	GET <code>/jobs/mine/applications</code> (SQLite activity store)
Admin	<code>/admin/console</code>	Monitor agents, run matches, inspect fairness	GET <code>/agents/status</code> , <code>/agents/events/recent</code> , <code>/system/config</code> ; optional <code>/match/*</code> and <code>/system/fairness</code>

All authenticated routes use cookie sessions (with `Credentials`) and FastAPI `require_role` guards. Match endpoints invoke the Matchmaking agent read-only over snapshots.

Cross-encoder reranking.. Optional second stage (`cross_encoder_rerank.py`, pool default 20) is disabled in production settings (`enable_cross_encoder_rerank=false`).

Explainability.. `match_explanation.py` builds structured payloads; `explain.py` adds rule-based bullets. Fit labels (Strong, Good, Moderate, Weak) derive from score bands documented in §5.

Fairness utilities.. `fairness.py` computes proxy-group selection rates; `benchmarks/fairness_audit.py` runs synthetic counterfactual tests offline.

Calibration and feedback.. Platt parameters persist in `data/models/calibration.json` ($a \approx 2.078$, $b \approx 2.112$). Feedback boost deltas: save +0.04, apply +0.06, dismiss -0.06 per historical action when enabled.

Fig. 5: Quality evaluation pipeline: labeled corpus, ranking drivers, metric aggregation, fairness audits, and regression gates against committed expected values.

4.6 Role portals and user experience

One React SPA mounts three guarded route trees. **Candidate portal:** onboarding, profile editor, job matches with explanation cards and save/apply actions. **Employer portal:** job list, reverse candidate match, applicant tracking. **Admin console:** agent status, event tail, fairness summary, manual match triggers for debugging.

Parsing panels show extracted resume or job fields before the user confirms; quality panels warn about missing skills or incomplete budgets. The matches view refreshes when a `PROFILE.UPDATED` event propagates after profile save. Client-side filters (e.g., remote-friendly) read explanation fields rather than duplicating constraint logic.

Users always act explicitly on matches; the system does not auto-apply or auto-reject.

4.7 Offline benchmark integration

Research drivers (`comparison.py`, `ablation.py`, `paper_progression.py`, `table11_fusion.py`, `negative_mining.py`) call the same scoring core with frozen corpus files under `data/`. Regression test `test_eval_regression.py` compares live runs against `data/expected/paper_progression_summary.json` with ± 0.04 nDCG tolerance. *TODO:* Commit `phase11_summary.csv` from `benchmarks/phase11.py` for ANN shortlist latency.

5 Quality Metrics

This section defines the scores used inside the Matchmaking agent and the offline metrics used to evaluate ranking quality, fairness sanity checks, and reproducibility. Implementation paths appear in §4; measured results in §6.

5.1 Evaluation corpus

We evaluate *resume-to-jobs* retrieval on a demo corpus: 30 candidate resumes (queries), 15 job postings, and 47 graded relevance pairs in `data/eval_pairs.json` (scale $r \in \{0, 1, 2\}$: none, partial, strong). Each query has one or two relevant jobs (21 strong, 26 partial labels). Table 2 summarizes statistics. Snapshots are built with the same agent code path used in production.

5.2 Similarity and component scores

All component scores are normalized to $[0, 1]$. Let $\mathbf{e}_c, \mathbf{e}_j \in \mathbb{R}^d$ denote snapshot embeddings ($d=384$).

Table 2: Evaluation corpus statistics.

Statistic	Value
Task	Resume $\rightarrow$ jobs (candidate-initiated retrieval)
Candidates (queries)	30
Job postings (corpus)	15
Graded query-document pairs	47
Queries with ≥ 1 relevant job	30
Relevance scale	0 (none), 1 (partial), 2 (strong)
Grade-2 (strong) labels	21
Grade-1 (partial) labels	26
Labels per query (min / max)	1 / 2
Embedding model	all-MiniLM-L6-v2 ($d=384$)
Label source	data/eval_pairs.json v1.0

Starter labeled set for offline benchmarks; not derived from live portal traffic.

Semantic similarity.. Cosine score:

$$s_{\text{sem}}^{\text{cos}} = \frac{\mathbf{e}_c \cdot \mathbf{e}_j}{\|\mathbf{e}_c\| \|\mathbf{e}_j\|}. \quad (1)$$

Euclidean-derived alternative: $s_{\text{sem}}^{\text{euc}} = 1/(1 + \|\mathbf{e}_c - \mathbf{e}_j\|_2)$.

Skill overlap.. For canonical skill sets A (resume) and B (required job skills), Jaccard mode:

$$s_{\text{skl}}^{\text{jac}} = |A \cap B|/|A \cup B| \quad (0 \text{ if either set empty}). \quad (2)$$

Soft-embed mode averages, over each $b \in B$, the maximum cosine between skill phrase embeddings.

Composite score (portal default)..

$$s_{\text{final}} = \sum_k w_k s_k, \quad (w_{\text{sem}}, w_{\text{skl}}, w_{\text{tit}}, w_{\text{exp}}, w_{\text{comp}}, w_{\text{rem}}) = (0.28, 0.27, 0.10, 0.15, 0.10, 0.10), \quad (3)$$

with s_k the semantic, skill, title, experience, compensation, and remote components; result clipped to $[0, 1]$.

Multimodal baseline.. $s_{\text{mm}} = \alpha s_{\text{sem}} + (1 - \alpha) s_{\text{skl}}$ with default $\alpha=0.7$.

Reciprocal rank fusion.. For ranked lists $\mathcal{L}_1, \dots, \mathcal{L}_m$:

$$\text{RRF}(x) = \sum_{i=1}^m \sum_r \mathbb{I}[\mathcal{L}_i[r] = x] \cdot \frac{1}{k + r}, \quad (4)$$

with $k=60$ by default.

Optional adjustments.. Hard constraints multiply s_{final} by feasibility factor $\phi \in (0, 1]$ when enabled. Platt calibration: $\hat{p}(s) = \sigma(as + b)$ with fitted (a, b) . Feedback boost adds weighted save/apply/dismiss deltas when enabled (off in default portal). *TODO:* Report calibration error (ECE or Brier) on held-out eval pairs. *TODO:* Report feedback-boost and calibration-on vs. calibration-off deltas on the eval corpus.

5.3 Ranking metrics

Given ranked list π_q for query q and relevant set $\mathcal{R}_q = \{j : \text{rel}_q(j) > 0\}$, with top- K prefix $\pi_q[:K]$ and default $K=5$:

$$\text{P@}K(q) = \frac{1}{K} \sum_{j \in \pi_q[:K]} \mathbb{1}[j \in \mathcal{R}_q], \quad (5)$$

$$\text{R@}K(q) = \frac{1}{|\mathcal{R}_q|} \sum_{j \in \pi_q[:K]} \mathbb{1}[j \in \mathcal{R}_q], \quad (6)$$

Graded nDCG:

$$\text{nDCG@}K(q) = \frac{\sum_{i=1}^K (2^{\text{rel}_q(j_i)} - 1) / \log_2(i + 1)}{\text{DCG@}K(q^*)}, \quad (7)$$

where π_{q^*} is the ideal ranking. MRR and MAP treat relevance as binary ($r > 0$). Metrics are macro-averaged over queries; significance tests use 5000 bootstrap resamples (seed 42).

5.4 Fairness and hard-negative diagnostics

Proxy-group audit.. Experience tier and remote-preference groups; reports selection-rate parity and disparate-impact ratio. *TODO*: Persist `fairness_eval.py` proxy-group ratios to `benchmark_outputs/`.

Synthetic counterfactual audit.. Ten fabricated profile pairs; flags rank or score instability under controlled field edits (Table 11 in §6).

Hard-negative mining.. From top- $P=10$ multimodal pool, up to five unlabeled high-scoring jobs per query:

$$\mathcal{H}_q = \{j \in \pi_q[:P] : j \notin \mathcal{R}_q\}_{|\cdot| \leq 5}. \quad (8)$$

Report: 30 queries, 150 pairs, zero overlap with declared relevant jobs.

5.5 Benchmark drivers and regression gates

Drivers share `eval_rankings()` but vary ranking strategy: comparison study (lexical vs. embedding), composite ablation, fusion ablation (`table11_fusion.py`), paper progression ladder, and optional ANN latency sweeps (`phase11.py`). *TODO*: No committed `phase11` summary artifact at time of writing.

Table 3 and Table 5 list method comparison and CI regression baselines; full numeric results appear in §6.

Regression gate (`test_eval_regression.py`):

$$\left| \widehat{\text{nDCG@}5_{\text{run}}} - \text{nDCG@}5_{\text{expected}} \right| \leq 0.04. \quad (9)$$

Table 3: Retrieval method comparison (macro-averaged, $K=5$).

Method	Family	P@5	R@5	MRR	nDCG@5	MAP
Multimodal weighted	Embedding	0.287	0.933	0.961	0.924	0.867
RRF ensemble	Embedding	0.293	0.950	0.944	0.913	0.845
TF-IDF cosine	Lexical	0.293	0.950	0.918	0.898	0.842
BM25	Lexical	0.307	0.983	0.912	0.894	0.838
Semantic cosine	Embedding	0.267	0.867	0.931	0.878	0.810
Semantic euclidean	Embedding	0.267	0.867	0.931	0.878	0.810
Soft skill embed	Embedding	0.280	0.900	0.911	0.869	0.829
Exact skill overlap	Lexical	0.233	0.733	0.816	0.748	0.681
Skills Jaccard	Embedding	0.233	0.733	0.816	0.748	0.681

Source: research pipeline run 2026-05-27T150000Z/comparison/comparison_summary.json.
Exhaustive ranking over all 15 jobs per query. Latency in Table 9.

Table 4: Paper progression benchmark (regression base-lines, $K=5$).

Method	P@5	R@5	nDCG@5
Multimodal soft embed w=0.7	0.313	1.000	0.969
Soft embed + cross-encoder (pool=10)	0.307	0.983	0.939
RRF ensemble (4 lists)	0.293	0.950	0.935
Multimodal Jaccard w=0.7	0.293	0.950	0.933
Semantic cosine (rich templates)	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901

Source: data/expected/paper_progression_summary.json. Produced by benchmarks/paper_progression.py; gated by tests/benchmarks/test_eval_regression.py (± 0.04 nDCG tolerance). Values can differ from Table 3 when template or cache flags differ.

Verified snapshot values (2026-05-27 artifacts, research driver config)..

Multimodal weighted ($\alpha=0.7$): nDCG@5 0.924; RRF ensemble: 0.913; full composite ablation (legacy weights 40/30/15/10/5): 0.942. These may differ from portal defaults; see §6 for interpretation. *TODO*: Re-run ablation with portal weights 28/27/10/15/10/10 and update Table 8.

6 Results and Discussion

We report macro-averaged ranking metrics at $K=5$ on the corpus defined in §5. All numbers come from committed benchmark artifacts; we do not extrapolate to production hiring outcomes. Results appear first; subsections thereafter interpret findings, state limits, and connect them to the multi-agent platform goals.

Table 5: Paper progression benchmark (regression baselines, $K=5$).

Method	P@5	R@5	nDCG@5
Multimodal soft embed $w=0.7$	0.313	1.000	0.969
Soft embed + cross-encoder (pool=10)	0.307	0.983	0.939
RRF ensemble (4 lists)	0.293	0.950	0.935
Multimodal Jaccard $w=0.7$	0.293	0.950	0.933
Semantic cosine (rich templates)	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901

Source: `data/expected/paper_progression.summary.json`. Produced by `benchmarks/paper_progression.py`; gated by `tests/benchmarks/test_eval_regression.py` (± 0.04 nDCG tolerance). Values can differ from Table 3 when template or cache flags differ.

Table 6: Fusion and constraint variants ($K=5$).

Method	P@5	R@5	nDCG@5
Learned fusion (LR)	0.307	0.983	0.968
Fixed multimodal ($\alpha=0.7$)	0.287	0.933	0.924
Hierarchical skills	0.287	0.933	0.924
Multimodal + constraints	0.300	0.950	0.908
Semantic cosine	0.267	0.867	0.878

Source: `backend/benchmark_outputs/table11.fusion.json` (`benchmarks/table11.fusion.py`).

6.1 Progression of retrieval strategies

Table 5 summarizes the paper-progression driver (`paper_progression.py`), which exhaustively ranks all 15 jobs per query. Among configurations in `data/expected/paper_progression.summary.json`, multimodal soft-embed ($\alpha=0.7$) achieves the highest nDCG@5 (0.969) and recall@5 (1.000). Lexical baselines (TF-IDF, BM25) remain competitive (nDCG@5 $\approx$ 0.901–0.905), indicating that keyword overlap still matters when the job pool is small.

RRF over four lists reaches nDCG@5 0.935, above semantic cosine (0.911) but below the best multimodal run. Cross-encoder reranking on pool=10 raises nDCG@5 to 0.939—still below soft-embed without reranking.

6.2 Fusion and constraint variants

Table 6 reports the fusion driver (`table11.fusion.py`). Learned logistic-regression fusion reaches nDCG@5 0.968 (P@5 0.307, R@5 0.983), within 0.001 nDCG of the best soft-embed progression run. Fixed multimodal ($\alpha=0.7$) and hierarchical blending tie at nDCG@5 0.924. Post-hoc constraint filtering lowers nDCG@5 to 0.908.

Table 7: Fitted calibration and fusion model parameters (offline training).

Component	Value
<i>Platt calibrator</i> (<code>data/models/calibration.json</code>)	
a (scale)	2.078
b (shift)	2.112
<i>Learned fusion LR</i> (<code>data/models/fusion.json</code>)	
Top feature weights (logistic)	exp. 0.860, remote 0.853, salary 0.849 semantic 0.537, taxonomy 0.509, skills 0.204 must-have coverage 0.311, bias 1.529
Models trained on labeled eval pairs; used when <code>use_calibration</code> or <code>fusion_mode=learned</code> is enabled. <i>TODO</i> : Report calibration impact on nDCG@5 and expected calibration error.	

Table 8: Ablation over composite matching components ($K=5$).

Variant	Category	P@5	R@5	MRR	nDCG@5	MAP
Full composite	Full	0.307	0.983	0.944	0.942	0.896
Semantic + skills	Partial	0.287	0.933	0.961	0.917	0.867
Semantic + skills + exp.	Partial	0.287	0.933	0.961	0.917	0.866
Semantic only	Single	0.267	0.867	0.931	0.878	0.810
Skills only	Single	0.233	0.733	0.816	0.748	0.681
Compensation only	Single	0.187	0.567	0.457	0.393	0.387
Location only	Single	0.167	0.467	0.388	0.335	0.328
Experience only	Single	0.160	0.467	0.384	0.326	0.314
Component RRF	Ensemble	0.233	0.717	0.570	0.564	0.497

Source: 2026-05-27T150000Z/ablation/ablation_summary.json. Weights for full composite in this run: semantic 40%, skills 30%, experience 15%, compensation 10%, location 5% (no title term). *TODO*: Re-run with portal weights 28/27/10/15/10/10.

Table 7 lists fitted calibration and fusion parameters ($a=2.078$, $b=2.112$ for Platt scaling).

6.3 Composite ablation and latency

Table 8 ablates composite components under research weights (40/30/15/10/5, no title term). Full composite reaches nDCG@5 0.942 vs. 0.917 for semantic+skills and 0.878 for semantic-only. Single constraint channels alone score $\text{nDCG@5} \leq 0.393$.

Table 9 reports latency: multimodal weighted ~ 0.26 ms/query over 15 jobs; soft skill embedding ~ 15.5 ms/query; cross-encoder rerank ~ 142 ms/query with nDCG@5 falling from 0.942 to 0.834 ($\Delta=-0.108$).

6.4 Hard-negative mining

Table 10: 30 queries, 150 hard-negative pairs, zero overlap with declared relevant jobs. Negatives are high-scoring *unlabeled* jobs, not confirmed irrelevants.

Table 9: Quality-latency trade-off (exhaustive evaluation, $K=5$).

Method	nDCG@5	Latency (ms/q)	Notes
Exact skill overlap	0.748	0.03	Lexical
Skills Jaccard	0.748	0.03	Set overlap
TF-IDF cosine	0.898	0.07	Lexical
BM25	0.894	0.11	Lexical
Semantic euclidean	0.878	0.21	Embedding
Semantic cosine	0.878	0.23	Embedding
Multimodal weighted	0.924	0.26	$\alpha=0.7$, Jaccard skills
RRF ensemble	0.913	0.95	Four ranked lists
Soft skill embed	0.869	15.53	Per-skill embedding
Composite (bi-encoder)	0.942	0.41	Ablation full composite
Composite + cross-encoder	0.834	141.7	Pool=20; Δ nDCG=-0.108

Latency from `comparison.summary.json` and `cross.encoder.report.json`. *TODO:* Publish `phase11.py` ANN shortlist latency table (pool=10, Chroma/Qdrant grid).

Table 10: Hard-negative mining summary (multimodal top-10 pool, $K=5$ negatives per query).

Statistic	Value
Queries mined	30
Negatives per query (target / observed)	5 / 5
Total hard-negative pairs	150
Negatives with graded relevance > 0	0
Negatives overlapping declared relevant set	0

Source: `backend/benchmark.outputs/hard_negatives.json`. Mined with multimodal weighted scoring ($\alpha=0.7$) over 15 jobs; negatives are high-scoring non-labeled jobs, not confirmed irrelevant documents.

6.5 Fairness audit

Table 11: synthetic counterfactual audit flags 6/10 fabricated pairs; max score shift 0.024. Engineering sanity check only—not demographic fairness validation.

6.6 Which strategy should the platform use?

Research drivers favor multimodal soft-embed (nDCG@5 0.969) and learned fusion (0.968), while the live portal defaults to composite weighting for interpretability (28/27/10/15/10/10). We do not claim one universally dominant strategy on this corpus. Combining semantic and skill signals consistently beats semantic-only retrieval; hard post-filter constraints hurt graded nDCG. *TODO:* Publish composite ablation under portal weights for deployment alignment.

Table 11: Synthetic counterfactual fairness audit (10 pairs, composite ranking).

Pair	Field changed	Top-1 stable	Top-5 overlap	Max rank Δ	Max score Δ	Flagged
name_gender_01	name	Yes	5/5	1	0.0096	No
name_gender_02	name	Yes	5/5	0	0.0072	No
name_ethnicity_01	name	Yes	5/5	2	0.0157	Yes
name_ethnicity_02	name	Yes	5/5	3	0.0238	Yes
nationality_phrase_01	summary	No	5/5	1	0.0087	Yes
nationality_phrase_02	summary	Yes	5/5	1	0.0110	No
hometown_01	hometown	Yes	4/5	2	0.0117	Yes
hometown_02	hometown	Yes	5/5	2	0.0143	Yes
pronouns_01	pronouns	Yes	5/5	1	0.0042	No
email_domain_01	email	Yes	4/5	1	0.0082	Yes

Source: docs/research/evaluation/paper_tables/table4_fairness.csv. Fabricated profiles only; 6/10 pairs flagged.
TODO: Export proxy-group disparate-impact ratios from fairness_eval.py.

6.7 Fusion stability and overfitting risk

RRF diversifies rankings without beating the best single multimodal run on this small corpus. Learned fusion ties soft-embed offline but trains on the same 47 labeled pairs used for evaluation—an exploratory upper bound, not a validated upgrade.

6.8 Latency–quality trade-offs

Lightweight scorers dominate on a 15-job pool. Cross-encoder reranking adds two orders of magnitude latency for lower nDCG in the committed report—consistent with keeping it disabled in default portal settings. *TODO:* Publish phase11.py ANN shortlist latency at larger pool sizes.

6.9 Implications for the multi-agent design

Offline metrics validate the Matchmaking agent’s scoring core under manual labels. They do not test whether event-driven cache invalidation or role-separated portals improve real hiring outcomes. The defensible claim is *bounded automation*: agents synchronize profiles and suggest explainable matches; users decide what to do next.

6.10 Limitations

- Small demo corpus (30 queries, 15 jobs); high recall@5 narrows nDCG spreads.
- Manual and synthetic labels, not ATS-sampled judgments.
- No production hiring outcomes measured.
- Fairness scope limited to synthetic counterfactuals; proxy-group ratios uncommitted.
- Configuration drift across drivers (portal vs. ablation weights).
- Learned fusion and calibration fit on eval data shared with test queries.

In sum, benchmarks support composite, explainable matchmaking on a controlled corpus. They do not yet justify strong claims about fairness, efficiency, or hiring quality at scale.

7 Conclusion and Future Scope

JobMatch demonstrates that hiring support can be organized as a **multi-agent recruitment system** rather than a monolithic search box. Candidate and Employer agents own profile state and shared embeddings; a read-only Matchmaking agent scores and explains candidate–job pairs; role-separated portals keep human users in control of applications and shortlists.

Offline evaluation on 30 resumes, 15 jobs, and 47 graded pairs shows that multi-modal soft-embed and learned fusion reach the highest nDCG@5 values in research drivers (0.969 and 0.968), while the production portal favors a fixed composite strategy for interpretability. Cross-encoder reranking reduces nDCG@5 by 0.108 at substantially higher latency; hard-negative mining yields 150 pairs with zero label conflicts; a synthetic fairness audit flags 6/10 fabricated pairs. These results are reproducible from committed benchmark artifacts but do not establish production hiring effectiveness.

Future scope. Larger held-out evaluation on the research corpus (data/research/: 100×50, 5000 labels) with portal-weight composite ablation. ANN latency artifacts from `phase11.py` to separate retrieval from scoring cost at scale. Committed proxy-group fairness exports and explanation-quality tables. Optional studies of calibration and feedback boost with metrics reported under §5. Field studies of recruiter and candidate trust in agent-mediated explanations remain outside this artifact but are necessary before operational deployment claims.

TODO: Run composite ablation with portal weights 28/27/10/15/10/10. *TODO:* Commit `phase11.summary.csv` and `fairness_eval` JSON exports.

The platform’s contribution is an integrated, explainable, multi-agent research prototype with traceable evaluation—a foundation for broader-audience hiring tools that show their work, not just their scores.

Acknowledgments. The authors thank collaborators who reviewed portal workflows and benchmark reproducibility checks.

Declarations

Funding

Not applicable.

Competing interests

The authors declare no competing interests.

Ethics approval

Not applicable. Evaluation uses synthetic and manually curated demo profiles; no human subjects recruitment data were collected for this study.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The demo corpus (`data/cvs.json`, `data/jobs.json`, `data/eval_pairs.json`) and benchmark outputs under `backend/benchmark_outputs/` and `docs/research/evaluation/` are included in the project repository. Regenerate metrics with `python backend/scripts/run_research_pipeline.py` and `python -m benchmarks.paper_progression`.

Code availability

Source code for the platform, benchmark drivers, and regression tests is available in the JobMatch repository at the submission tag. Key entry points: `backend/gateway/`, `backend/agents/`, `frontend/src/`, and `tests/benchmarks/test_eval_regression.py`.

Author contributions

First Author: conceptualization, software, evaluation, writing—review and editing. Additional authors: *TODO*: Add CRediT roles before submission.

Prior publication

This manuscript has not been published previously and is not under consideration elsewhere. No portion of the text has appeared in a preprint with colored layout styling.

References

- [1] Wooldridge, M.: An Introduction to MultiAgent Systems, 2nd edn. Wiley, ??? (2009)
- [2] Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, ??? (2008)

- [3] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. In: Foundations and Trends in Information Retrieval, vol. 3, pp. 333–389 (2009)
- [4] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)
- [5] Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Proceedings of SIGIR, 758–759 (2009)
- [6] Ribeiro, M.T., Singh, S., Guestrin, C.: “why should i trust you?”: Explaining the predictions of any classifier. Proceedings of KDD, 1135–1144 (2016)
- [7] Platt, J.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 61–74 (1999)